

Now you've loaded your data, built an index, and stored that index for later, you're ready to get to the most significant part of an LLM application: querying.

At its simplest, querying is just a prompt call to an LLM: it can be a question and get an answer, or a request for summarization, or a much more complex instruction.

More complex querying could involve repeated/chained prompt + LLM calls, or even a reasoning loop across multiple components.

Getting started

The basis of all querying is the `QueryEngine`. The simplest way to get a `QueryEngine` is to get your index to create one for you, like this:

```
query_engine = index.as_query_engine()
response = query_engine.query(
    "Write an email to the user given their background information."
)
print(response)
```

Stages of querying

However, there is more to querying than initially meets the eye. Querying consists of three distinct stages:

- **Retrieval** is when you find and return the most relevant documents for your query from your `Index`. As previously discussed in [indexing](#), the most common type of retrieval is “top-k” semantic retrieval, but there are many other retrieval strategies.
- **Postprocessing** is when the `Nodes` retrieved are optionally reranked, transformed, or filtered, for instance by requiring that they have specific metadata such as keywords attached.
- **Response synthesis** is when your query, your most-relevant data and your prompt are combined and sent to your LLM to return a response.

Tip

You can find out about [how to attach metadata to documents](#) and [nodes](#).

Customizing the stages of querying

LlamaIndex features a low-level composition API that gives you granular control over your querying.

In this example, we customize our retriever to use a different number for `top_k` and add a post-processing step that requires that the retrieved nodes reach a minimum similarity score to be included. This would give you a lot of data when you have relevant results but potentially no data if you have nothing relevant.

```
from llama_index.core import VectorStoreIndex, get_response_synthesizer
from llama_index.core.retrievers import VectorIndexRetriever
from llama_index.core.query_engine import RetrieverQueryEngine
from llama_index.core.postprocessor import SimilarityPostprocessor

# build index
index = VectorStoreIndex.from_documents(documents)

# configure retriever
retriever = VectorIndexRetriever(
    index=index,
    similarity_top_k=10,
)

# configure response synthesizer
response_synthesizer = get_response_synthesizer()

# assemble query engine
query_engine = RetrieverQueryEngine(
    retriever=retriever,
    response_synthesizer=response_synthesizer,
    node_postprocessors=[SimilarityPostprocessor(similarity_cutoff=0.7)],
)

# query
response = query_engine.query("What did the author do growing up?")
print(response)
```

You can also add your own retrieval, response synthesis, and overall query logic, by implementing the corresponding interfaces.

For a full list of implemented components and the supported configurations, check out our [reference docs](#).

Let's go into more detail about customizing each step:

Configuring retriever

```
retriever = VectorIndexRetriever(  
    index=index,  
    similarity_top_k=10,  
)
```

There are a huge variety of retrievers that you can learn about in our [module guide on retrievers](#).

Configuring node postprocessors

We support advanced `Node` filtering and augmentation that can further improve the relevancy of the retrieved `Node` objects. This can help reduce the time/number of LLM calls/cost or improve response quality.

For example:

- `KeywordNodePostprocessor` : filters nodes by `required_keywords` and `exclude_keywords`.
- `SimilarityPostprocessor` : filters nodes by setting a threshold on the similarity score (thus only supported by embedding-based retrievers)
- `PrevNextNodePostprocessor` : augments retrieved `Node` objects with additional relevant context based on `Node` relationships.

The full list of node postprocessors is documented in the [Node Postprocessor Reference](#).

To configure the desired node postprocessors:

```
node_postprocessors = [  
    KeywordNodePostprocessor(  
        required_keywords=["Combinator"], exclude_keywords=["Italy"]  
    )  
]  
query_engine = RetrieverQueryEngine.from_args(  
    retriever, node_postprocessors=node_postprocessors  
)  
response = query_engine.query("What did the author do growing up?")
```

Configuring response synthesis

After a retriever fetches relevant nodes, a `BaseSynthesizer` synthesizes the final response by combining the information.

You can configure it via

```
query_engine = RetrieverQueryEngine.from_args(  
    retriever, response_mode=response_mode  
)
```

Right now, we support the following options:

- `default` : “create and refine” an answer by sequentially going through each retrieved `Node` ; This makes a separate LLM call per `Node`. Good for more detailed answers.
- `compact` : “compact” the prompt during each LLM call by stuffing as many `Node` text chunks that can fit within the maximum prompt size. If there are too many chunks to stuff in one prompt, “create and refine” an answer by going through multiple prompts.
- `tree_summarize` : Given a set of `Node` objects and the query, recursively construct a tree and return the root node as the response. Good for summarization purposes.
- `no_text` : Only runs the retriever to fetch the nodes that would have been sent to the LLM, without actually sending them. Then can be inspected by checking `response.source_nodes` . The response object is covered in more detail in Section 5.
- `accumulate` : Given a set of `Node` objects and the query, apply the query to each `Node` text chunk while accumulating the responses into an array. Returns a concatenated string of all responses. Good for when you need to run the same query separately against each text chunk.

Structured Outputs

You may want to ensure your output is structured. See our [Query Engines + Pydantic Outputs](#) to see how to extract a Pydantic object from a query engine class.

Also make sure to check out our entire [Structured Outputs](#) guide.

Creating your own Query Workflow

If you want to design complex query flows, you can compose your own query workflow across many different modules, from prompts/LLMs/output parsers to retrievers to response synthesizers to your own custom components.

Take a look at our [Workflow Guide](#) for more details.